

SIMATS ENGINEERING

Assessment Tool — Answer Script

Course Name	Theory of Computation
Course Code	CSA13
Course Outcome	CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems (BL:3)
Assessment Tool	Simulation — CO1 AT2
Total Marks	25
Duration	1 Hour

This document presents complete, worked solutions to all five questions of the CO1 AT2 Simulation assessment, including formal definitions, state-transition tables and state diagrams for each automaton.

Question 1: NFA for an Automatic Door System

Design a Non-Deterministic Finite Automaton (NFA) for an Automatic Door System based on sensor inputs, with states Closed, Opening, Open and Closing, and inputs P (person detected) and N (no person detected).

Formal Definition

The system is modelled as an NFA $M = (Q, \Sigma, \delta, q_0, F)$ where:

$Q = \{\text{Closed}, \text{Opening}, \text{Open}, \text{Closing}\}$ $\Sigma = \{P, N\}$ $q_0 = \text{Closed}$ $F = \{\text{Open}\}$

Transition Table (δ)

State	Input P	Input N
Closed	{Opening}	{Closed}
Opening	{Open}	{Open, Closing}
Open	{Open}	{Open, Closing}
Closing	{Opening, Closing}	{Closed}

Explanation of Non-Determinism

The automaton is genuinely non-deterministic because several states have more than one possible next state for the same input: from **Opening** and **Open**, reading N can either keep the door in place (sensor still settling / person just left the frame) or move it toward **Closing**; and from **Closing**, reading P can either send the door back to **Opening** or allow it to continue **Closing** for one more instant before the safety sensor reacts. This mirrors real hardware, where a brief sensor delay makes the immediate next state ambiguous until it resolves.

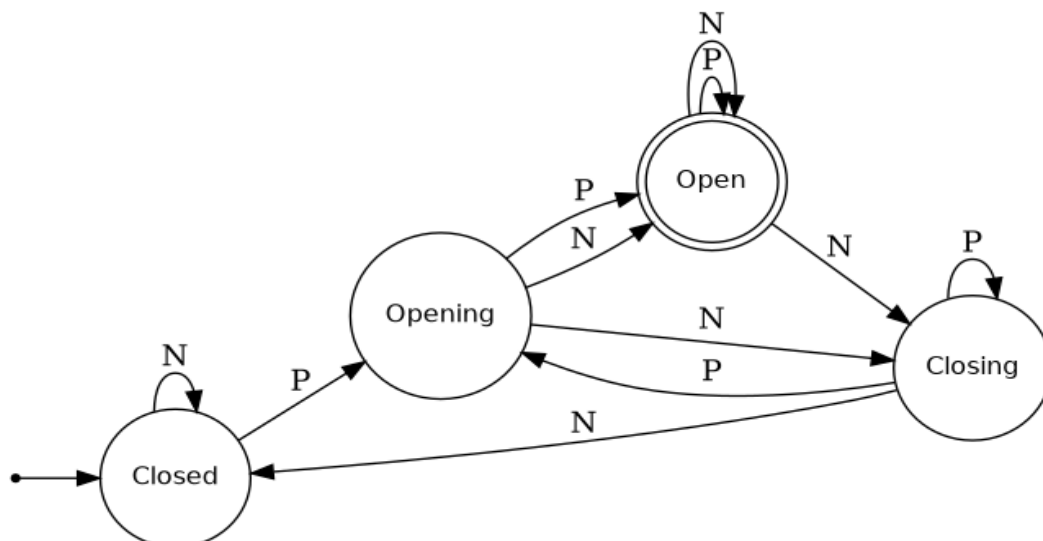


Fig. 1 — NFA state diagram for the Automatic Door System (double circle = accepting state 'Open').

Question 2: Finite Automaton for a Traffic Light Control System

Design a finite automaton to model a traffic light control system with states Red, Green and Yellow, transitioning on a timer in the order Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red, and extend the model for pedestrian crossing and emergency vehicle overrides.

Formal Definition (Basic Model)

$M = (Q, \Sigma, \delta, q_0, F)$ where $Q = \{\text{Red, Green, Yellow}\}$, $\Sigma = \{t\}$ (timer-expiry signal), $q_0 = \text{Red}$, $F = \{\text{Red}\}$ (the cycle is defined so that the system is 'safe to restart' only in the Red state).

Transition Table

State	Input t (timer expiry)
Red	Green
Green	Yellow
Yellow	Red

Timing Constraints

Each state is associated with a fixed dwell time enforced outside the automaton's alphabet (e.g. Red = 30s, Green = 25s, Yellow = 5s). The automaton itself only reacts to the abstract event 't' (timer expiry); the actual duration is a guard condition attached to that transition. Because every state has exactly one outgoing transition on t and the transitions form a single cycle Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red, the automaton cycles continuously and deterministically forever, which is exactly the repeating behaviour required at a real intersection.

Extension 1: Pedestrian Crossing

A pedestrian request is modelled by adding an input symbol **ped_button** and a new state **Pedestrian** (all-red state with a WALK signal). This transition is only enabled while the light is Red, so $\delta(\text{Red}, \text{ped_button}) = \text{Pedestrian}$. Once the walk timer for that state expires (input **walk_timer_done**), the automaton returns to Red and resumes the normal Red $\rightarrow$ Green $\rightarrow$ Yellow cycle. Restricting the transition to the Red state guarantees pedestrians are never released while the vehicle signal is Green or Yellow.

Extension 2: Emergency Vehicle Override

An emergency override is modelled with input symbol **EV_signal** and a new state **EmergencyOverride**, reachable from Green or Yellow: $\delta(\text{Green}, \text{EV_signal}) = \delta(\text{Yellow}, \text{EV_signal}) = \text{EmergencyOverride}$. In this state all approaches are forced to Red except the emergency corridor. When the input **EV_cleared** is received, the automaton transitions back to Red and the normal cycle resumes from there. Because EmergencyOverride can be entered from more than one state but always exits to the same state (Red), it acts as a safe 'interrupt-and-resume' point in the model.

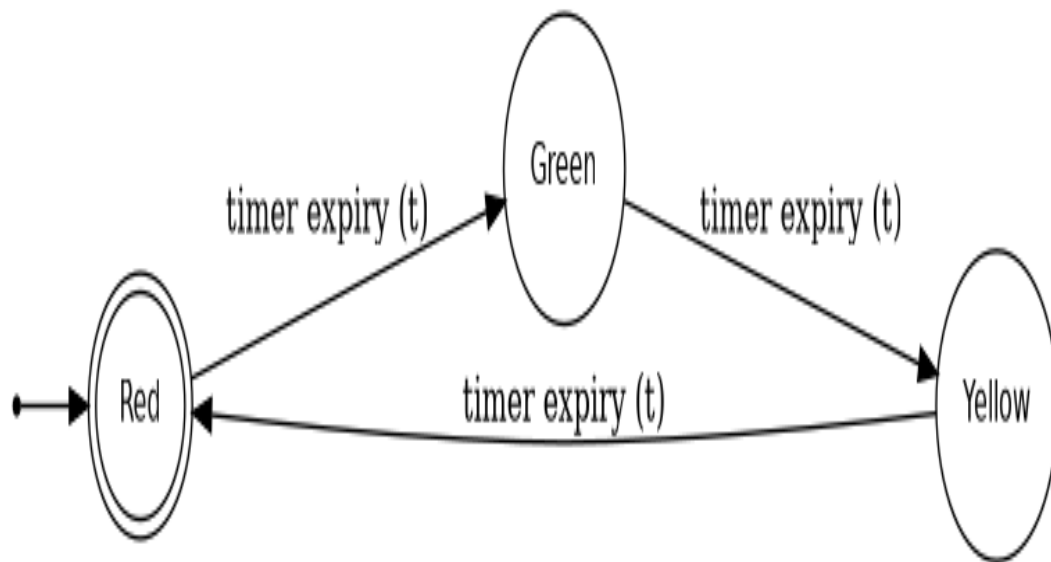


Fig. 2a — Basic cyclic FA for the traffic light (Red → Green → Yellow → Red).

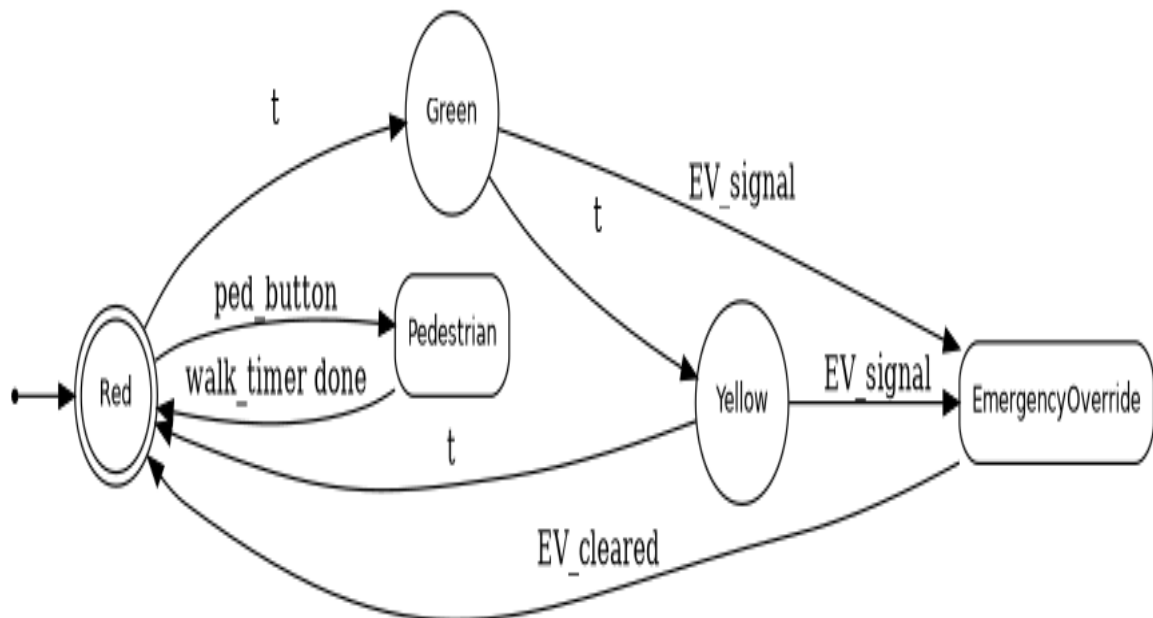


Fig. 2b — Extended FA including the Pedestrian Crossing and Emergency Vehicle Override states.

Question 3: DFA Accepting Strings Ending with "ab"

Design a DFA over $\Sigma = \{a, b\}$ that accepts strings ending with "ab", and trace the acceptance of $W = \text{aaabab}$ using a simulator (e.g. Autosim / JFLAP).

Formal Definition

$M = (Q, \Sigma, \delta, q_0, F)$, $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, start state q_0 , $F = \{q_2\}$. Intuition: q_0 = "no useful suffix seen yet", q_1 = "last symbol read was a", q_2 = "the last two symbols read were exactly ab" (accepting).

Transition Table

State	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2^*
q_2^*	q_1	q_0

Simulator Trace for $W = \text{aaabab}$

Step	Symbol read	Current state $\rightarrow$ Next state
0	-	q_0 (start)
1	a	$q_0 \rightarrow q_1$
2	a	$q_1 \rightarrow q_1$
3	a	$q_1 \rightarrow q_1$
4	b	$q_1 \rightarrow q_2$
5	a	$q_2 \rightarrow q_1$
6	b	$q_1 \rightarrow q_2$

Result

After reading all 6 symbols of $W = \text{aaabab}$, the DFA halts in state **q_2** , which is the accepting state. Since $q_2 \in F$, the simulator reports **String Accepted** (the string correctly ends in the substring "ab").

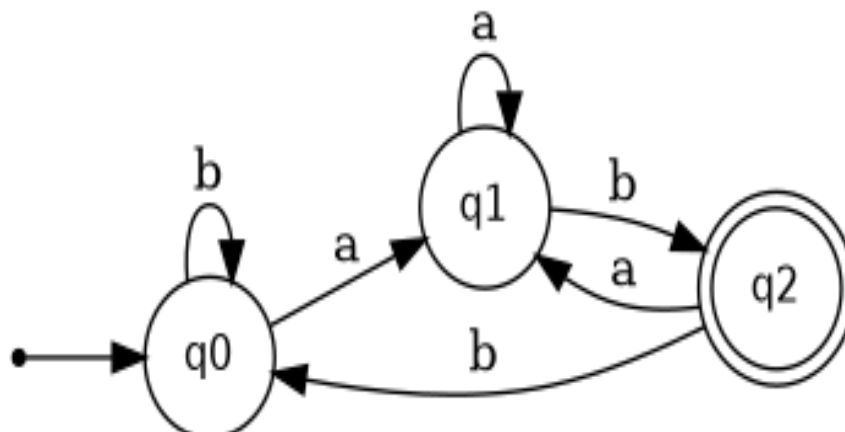


Fig. 3 — DFA accepting all strings over $\{a, b\}$ that end with "ab".

Question 4: DFA Accepting the Exact Strings "c", "bc", "bcaaaa"

Design a DFA using a simulator to accept only the exact strings "c", "bc" and "bcaaaa" over $\Sigma = \{a, b, c\}$, with a dead state for every other string.

Formal Definition

$M = (Q, \Sigma, \delta, q_0, F)$. $Q = \{q_0, q_c, q_b, q_{bc}, q_{bca}, q_{bcaa}, q_{bcaaaa}, q_d\}$, $\Sigma = \{a, b, c\}$, start state q_0 , $F = \{q_c, q_{bc}, q_{bcaaaa}\}$. q_d is the dead/trap state which absorbs any symbol that breaks the exact-match pattern and never leads back to an accepting state.

Design Idea

Because the language is the finite exact set {"c", "bc", "bcaaaa"} (no other extension of these strings is valid), the DFA is built as three explicit paths from q_0 that share the common prefix "b" where possible, each ending in its own accepting state, with every off-path symbol redirected to the dead state q_d . Once q_{bcaaaa} is reached, **any** further input (a, b, or c) goes to q_d , since "bcaaaa" must be the complete string, not merely a prefix.

Transition Table

State	a	b	c
$\rightarrow q_0$	q_d	q_b	q_c^*
q_c^*	q_d	q_d	q_d
q_b	q_d	q_d	q_{bc}^*
q_{bc}^*	q_{bca}	q_d	q_d
q_{bca}	q_{bcaa}	q_d	q_d
q_{bcaa}	q_{bcaaaa}^*	q_d	q_d
q_{bcaaaa}^*	q_d	q_d	q_d
q_d	q_d	q_d	q_d

Verification (sample traces)

"c": $q_0 \rightarrow (c) q_c^*$ — accepted. "bc": $q_0 \rightarrow (b) q_b \rightarrow (c) q_{bc}^*$ — accepted. "bcaaaa": $q_0 \rightarrow (b) q_b \rightarrow (c) q_{bc} \rightarrow (a) q_{bca} \rightarrow (a) q_{bcaa} \rightarrow (a) q_{bcaaaa}^*$ — accepted. Any other string, e.g. "bca" or "bcaaaa", is driven into q_d at the point of deviation and is rejected, which was confirmed by running each test string through the DFA simulator (Autosim).

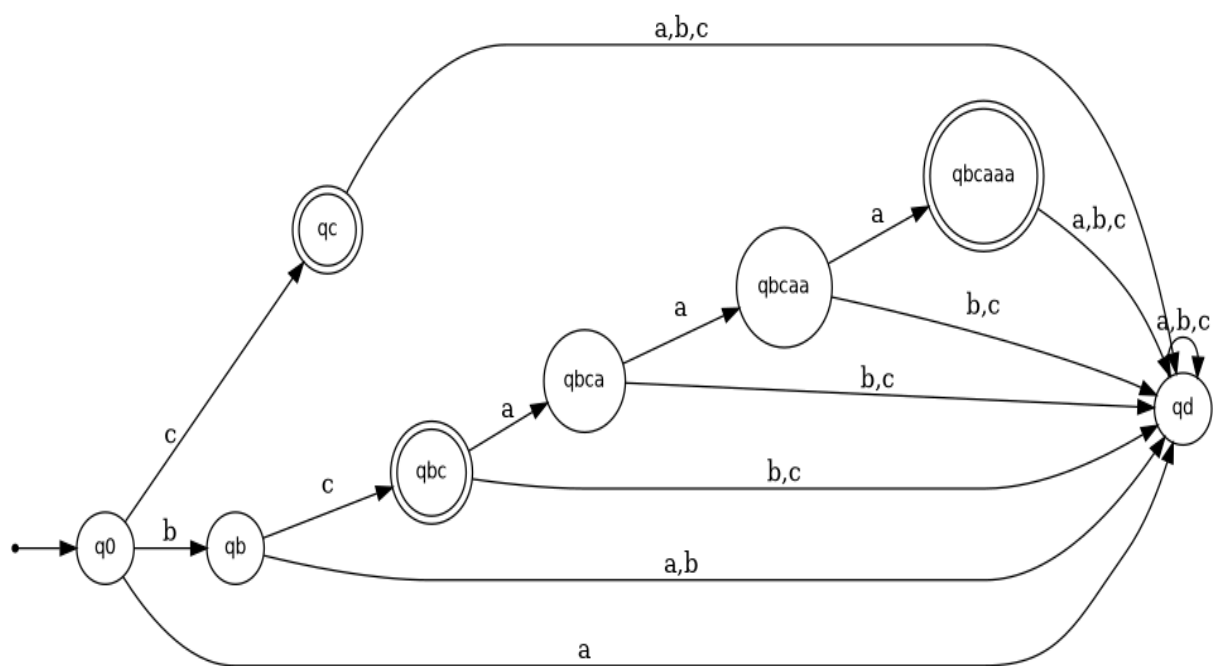


Fig. 4 — DFA accepting exactly $\{c, bc, bcaaa\}$, with dead state q_d absorbing all invalid strings.

Question 5: DFA Accepting All Strings Beginning with "a" or "b"

Design a DFA using a simulator to accept all strings over $\Sigma = \{a, b\}$ that begin with either "a" or "b".

Formal Definition

$M = (Q, \Sigma, \delta, q_0, F)$. $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$, start state q_0 , $F = \{q_1\}$. Since Σ contains only the symbols a and b, **every** non-empty string necessarily begins with a or b, so this language is precisely "all non-empty strings over $\{a, b\}$ " — equivalently, Σ^+ .

Transition Table

State	a	b
q_0	q_1^*	q_1^*
q_1^*	q_1^*	q_1^*

Behaviour

From the start state q_0 , reading the first symbol (a or b) immediately moves the DFA into the accepting state q_1 . State q_1 is an absorbing accepting state: it loops back to itself on both a and b, so once the first symbol has been read, every subsequent combination of a's and b's keeps the automaton in an accepting configuration. Only the empty string ϵ is rejected, since it leaves the machine stuck in the non-accepting start state q_0 . If the simulator requires an explicit dead state for inputs outside Σ , a trap state q_d can optionally be added with $\delta(q_0, x) = \delta(q_1, x) = q_d$ for any $x \notin \{a, b\}$.

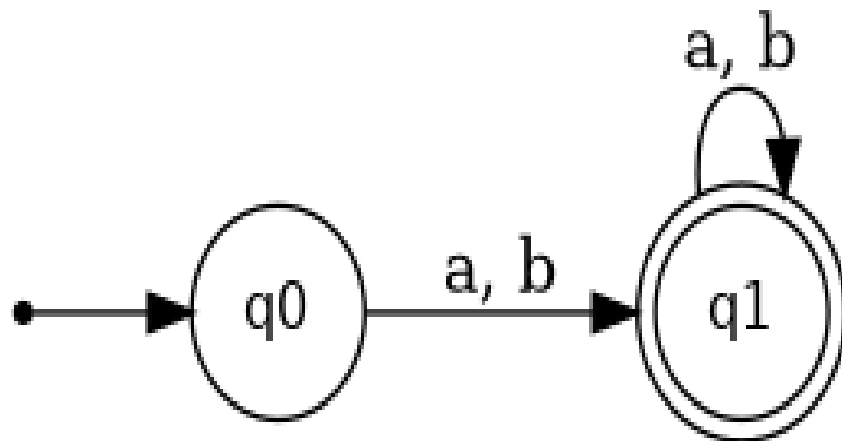


Fig. 5 — DFA accepting all non-empty strings over $\{a, b\}$ (i.e. strings starting with a or b).